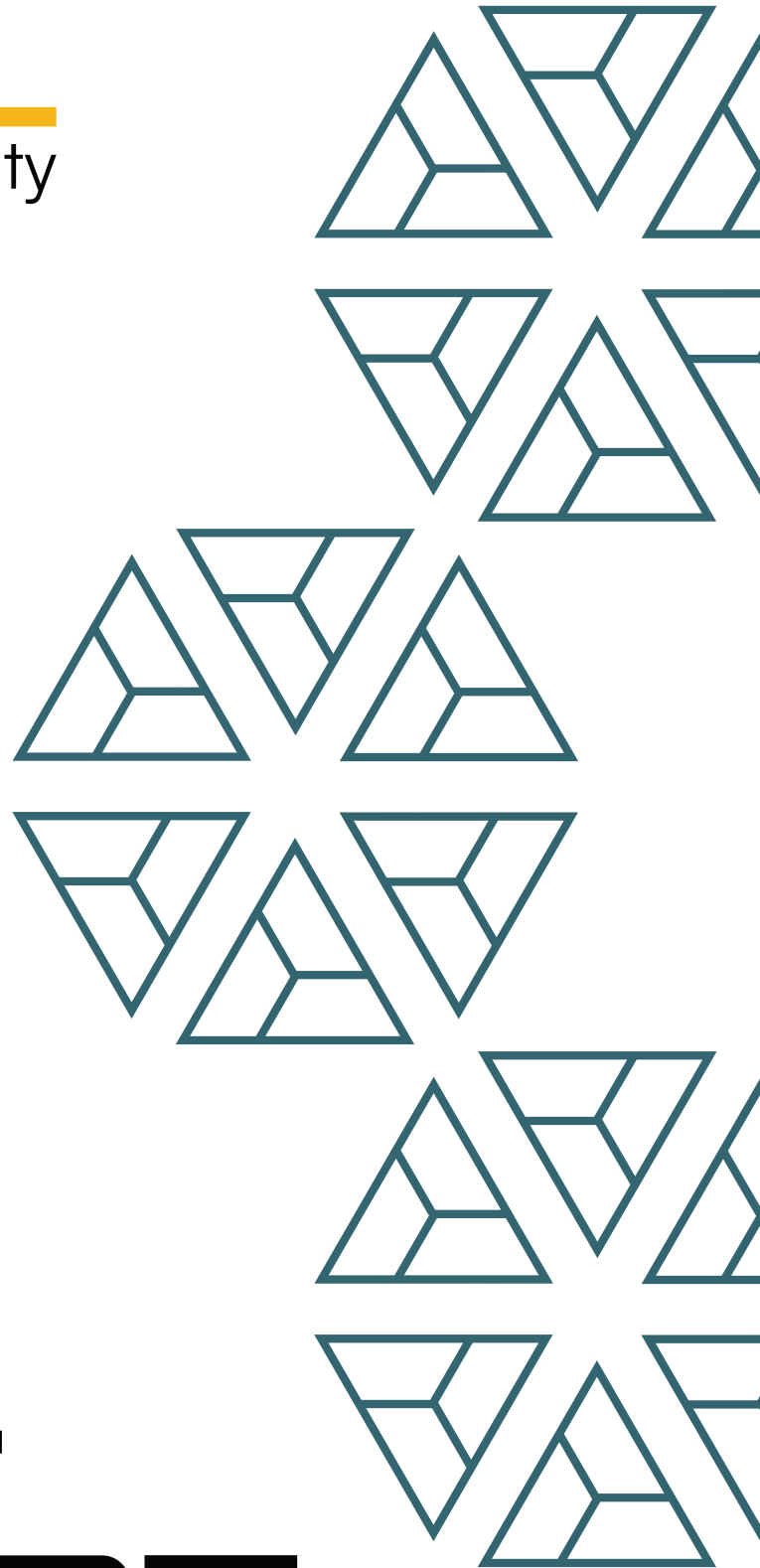




BAIL
security



Symbiotic
Adapters

FINAL REPORT

April '2026

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:

Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Symbiotic - Adapters - Audit Report
Website	symbiotic.fi
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/symbioticfi/core-mirror/tree/4770a884b76132e3b8a1106ca00e0a9dcf30c778/src/contracts/vault/adapters
Resolution 1	https://github.com/symbioticfi/core-mirror/tree/fea67c231cb41a0675fa70fad43153e10b2ba41b/src/contracts
Resolution 2	https://github.com/symbioticfi/core-mirror/tree/49cad3c7070ed2de6b91f9bf48695b14140fd6b8

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no changes made)	Failed resolution	Open
High	2	1	1			
Medium	4	2	2			
Low	16	4	1	11		
Informational	13	1		12		
Governance	2			2		
Total	37	8	4	25		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

Adapter

The **Adapter** contract is a simple abstract contract which is meant to be inherited by the **MorphoVaultV2Adapter** and **AAVEV3Adapter** contracts. It exposes the adapter base which includes a global allocation limit per collateral token and likewise keeps track of the global allocation per collateral token.

Privileged Functions

- `setGlobalLimit`

Core Invariants:

INV 1: Each collateral token cannot exceed its allocation beyond the `globalAllocated` value

Issue_01	Governance: Control over funds
Severity	Governance
Description	Currently, governance of this contract has several privileges for invoking certain functions that can drastically alter the contracts behavior. For example, the contract can be upgraded.
Recommendations	Consider incorporating a Gnosis Multisignature contract as owner and ensuring that the Gnosis participants are trusted entities.
Comments / Resolution	Acknowledged.

Issue_02	Beacon owner trust assumptions
Severity	Governance
Description	The beacon owner has the ability to upgrade the implementation used by all proxies. A malicious or compromised owner could upgrade to an implementation that drains funds from the proxies through various mechanisms.
Recommendations	Consider incorporating a Gnosis Multisignature contract as owner and ensuring that the Gnosis participants are trusted entities.
Comments / Resolution	Acknowledged.

Issue_03	Vault can be broken due to donation in edge-cases
Severity	High
Description	Please note that this issue technically belongs to the Vault V2 contract. However, since the relevant flow passes through the adapter contracts, we believe it is still worth mentioning here. VaultV2 integrates external adapters whose positive excess value is harvested through adapter <code>skim()</code> functions and then routed into <code>Rewards.distributeDonationRewards(vault, amount)</code>. The rewards contract subsequently calls <code>VaultV2.donate(amount)</code>, which redistributes the donated amount across the vault's current <code>activeStake</code> and <code>activeWithdrawals</code>. Importantly, matured but unclaimed withdrawals are no longer counted inside <code>activeWithdrawals()</code>. Also, many important vault operations force skimming or deallocation paths that begin with skimming, including deposit, withdraw, redeem, claim, <code>instantWithdraw</code>, public <code>deallocateAdapters()</code>, and slash-related recall paths. The issue occurs when the vault reaches a state where both <code>activeStake</code> and <code>activeWithdrawals</code> are zero, but an adapter still has positive skimmable value. In that situation, a successful adapter skim triggers <code>Rewards.distributeDonationRewards(...)</code>, which eventually calls <code>VaultV2.donate(amount)</code>. <code>donate()</code> computes the withdrawal-side allocation using <code>curActiveStake + curActiveWithdrawals</code> as the denominator. If both are zero, the donation reverts due to division by zero. Because the vault forces skimming on core entry functions and adapter deallocation also begins with skimming, this does not only block rewards realization; it can block the vault's core user and liquidity-recall flows. The root cause is that the adapter yield-harvest path assumes there is always a non-zero active recipient set inside the vault, while the vault explicitly excludes matured/claimable withdrawals from <code>activeWithdrawals()</code>.

	As a result, claimable-but-not-yet-claimed users do not provide a valid donation denominator, and the system can deadlock once all active pools have gone to zero. ### This can be actively broken by an attacker as follows: Attacker deposits 100 tokens Attacker calls withdraw for 100 tokens; waits until matured no activeShares anymore Attacker donates yield to adapter any skim call will now revert ### Furthermore, it will also happen in case of 100% slashing.
Recommendations	Consider changing the logic within the VaultV2 contract. We also recommend audit coverage for this contract.
Comments / Resolution	Fixed, if there is no stake, yield is given to the protocol.

Issue_04	Idle collateral is allocated to first deallocated vault and will disrupt accounting state
Severity	Medium
Description	Please note that this issue is technically within the MorphoVaultV2Adapter and AAVEV3Adapter contracts but since the description is the same, it will be mentioned within this section. Both <code>AaveV3Adapter</code> and <code>MorphoVaultV2Adapter</code> maintain two distinct accounting layers. First, the vault and adapter track principal-like exposure through <code>VaultV2.adapterAllocated[adapter]</code>, <code>VaultV2.adaptersAllocated</code>, and <code>Adapter.globalAllocated[collateral]</code>. Second, the adapter tracks the vault's actual entitlement to the pooled external position through its internal share ledger (<code>vaultShares / total...Shares</code>) and derives the current claim with <code>_getVaultAssets[vault]</code>. During deallocation, the adapters treat raw underlying collateral already sitting idle in the adapter (<code>collateral.balanceOf(address[this])</code>) as immediately recallable and count it inside <code>deallocatable[vault]</code>. If that idle balance covers the requested deallocation, or if the external withdraw reverts and the adapter falls back to returning only its idle balance, the vault-side and adapter-side allocation ledgers are reduced even though the external position and the vault's internal adapter shares remain unchanged. The issue is that deallocation accounting is driven by the amount of collateral returned, not by whether the vault's actual external entitlement was reduced. This allows a vault to consume idle collateral that was never part of the external Aave/Morpho position and still have that amount deducted from <code>adapterAllocated</code> and <code>globalAllocated</code>. The same effect occurs if the external withdraw reverts and the adapter

returns only `curBalance` from its catch path.

In both cases, the vault's internal adapter shares are left untouched, so `_getVaultAssets[vault]` stays the same while recorded principal falls.

A simple reachable sequence is: a vault has a live external adapter position and positive internal adapter shares; someone directly transfers the underlying collateral token to the adapter (i.e. pending external Aave/Morpho incentives); the vault then deallocates using that idle collateral. After the call, `adapterAllocated` and `globalAllocated` are lower or even zero, but the vault still retains the same claim on the external position.

At that point the system starts treating the remaining external claim as if it were yield, because `skimmable[vault] = _getVaultAssets[vault] - adapterAllocated[vault]` has increased solely due to the stale-low allocation ledger. The root cause is that the allocation ledgers are decremented by idle collateral returned to the vault, while the entitlement ledger is decremented only when the external position is actually withdrawn and shares are burned.

A user who becomes aware of this behavior and notices idle collateral in the adapter contract could sandwich the yield that will be generated by the donated funds during deallocation by depositing a significant amount before the yield spike occurs and withdrawing after that yield has been redistributed to vault shareholders.

Impact 1: the first vault that deallocates can capture directly donated idle collateral as principal, even though that collateral is not attributed pro rata through the adapter share system

Impact 2: a vault can reach a state where `adapterAllocated == 0` while it still has a positive external entitlement, causing the full remaining external claim to be treated as skimmable yield and later routed through Rewards donation

Impact 3: `globalAllocated[collateral]` can become lower than the adapter's real external exposure, which increases `allocatable[]` and allows effective overbooking against the configured global cap (it is

	technically still correct in terms of “allocations”, depending on the POV. Impact 4: because <code>VaultV2.setAdapterLimit(adapter, 0)</code> and adapter removals are gated only by <code>adapterAllocated[adapter]</code>, the vault can remove or disable an adapter while real external value is still present, potentially stranding that value Impact 5: vault-side risk metrics such as <code>_adaptersOwe()</code> and adapter-exposure checks can become understated, because they read the stale-low allocation rather than the real external exposure
Recommendations	This issue is inherently tied to the fact that the idle balance is incorporated into the deallocation mechanism which is in fact required in case of fallback deallocation where funds cannot be allocated and are required to be skimmed back. Therefore, a change is non-trivial.
Comments / Resolution	Fixed by removing the idle balance from accounting and isolating each vault accounting into separate proxies.

Issue_05	Skim can be forced to revert which allows for stealing yield from users
Severity	Medium
Description	Please note that this issue is technically within the MorphoVaultV2Adapter and AAVEV3Adapter contracts but since the description is the same, it will be mentioned within this section. Whenever a deposit happens, adapters are skimmed before the <code>activeStakes</code> are increased. This is to ensure accrued yield cannot be stolen by incoming depositors. However, by making the third-party protocol withdrawal revert (ie. taking a flashloan), skim now reverts and <code>activeStakes</code> is increased which then grants the old accrued yield towards the new depositor.
Recommendations	There is no simple fix for this issue, consider implementing a keeper which regularly skims rewards, making sure the accrued yield will not grow too large without being skimmed. This will partially fix this issue.
Comments / Resolution	Partially resolved, now allocations revert when there is some amount to skimmable after <code>skim()</code> is executed. This is however not true for adapters which are not element zero.

Issue_06	Inflation attack possibility due to increase of share token balance in adapter
Severity	Medium
Description	Please note that this issue is technically within the MorphoVaultV2Adapter and AAVEV3Adapter contracts but since the description is the same, it will be mentioned within this section. It is possible to make the first deposit followed by a share token transfer (AToken/Morpho Vault share token) which then increases the denominator for new deposits, even after skimming funds the proportional decrease of shares will still result in an inflation attack vulnerable situation where subsequent depositors may receive zero shares or rounded down shares. Example: Step 1: Attacker makes first deposit (1 wei) Vault A allocates 1 wei to adapter: <pre> mintedShares = previewDeposit(1, 0, 0) = floor(1 × 1/1) = 1 aave.supply[1]: adapterAssets → 1 vaultShares[A] = 1 tcs = 1 adapterAssets = 1 adapterAllocated_A = 1 </pre> Step 2: Attacker donates aToken directly to adapter Attacker transfers 10000 aToken to adapter address: <pre> aToken.transfer(adapter, 10000) adapterAssets = 10001 (1 from supply + 10000 donated) vaultShares[A] = 1 tcs = 1 (unchanged; no allocate call) adapterAllocated_A = 1 (unchanged) </pre>

Step 3: Skim fires [e.g., on next vault operation]

```
skimmable[A] = _getVaultAssets[A] - adapterAllocated_A
_getVaultAssets = floor[1 × 10002/2] = 5001
skimmable = 5001 - 1 = 5000
burnedShares = ceil[5000 × 2/10002] = ceil[0.9998] = 1 ← ALL
shares burned
aave.withdraw(5000):
adapterAssets → 5001
vaultShares[A] = 0
tcs = 0
5000 distributed via REWARDS → vault.donate
```

Step 4: Victim deposits - gets zero shares

```
Vault B allocates 100 to the same adapter:
mintedShares = previewDeposit(100, 0, 5001)
= floor[100 × (0+1) / (5001+1)]
= floor[100/5002] = 0 ZERO SHARES
aave.supply(100):
adapterAssets → 5101
vaultShares[B] += 0
tcs += 0
```

Victim's 100 tokens were absorbed into the pool with no ownership.

Note that in this example, it demonstrates only the loss of tokens without any gain for the attacker, this depends at the end on the numeric values but it demonstrates the (non) impact of skim on the exchange rate. It also demonstrates how virtual shares will absorb funds which are allocated to nobody, disrupting the accounting.

Similarly, a user can also donate funds to the vault without any previous deposit existing which would simply allocate these funds to the virtual shares and will also influence subsequent deposits.

Recommendations	Consider implementing a zero share check, which partially fixes this issue. Increasing the virtual shares will increase the allocated dead amount which will result in other extended side-effects, see related issue.
Comments / Resolution	Fixed by removing the share mechanism.

Issue_07	Auto-allocation can be abused as piggy bank mechanism
Severity	Low
Description	Please note that this issue is technically within the MorphoVaultV2Adapter and AAVEV3Adapter contracts but since the description is the same, it will be mentioned within this section. The auto-allocation during deposits allows users to use adapters as piggy banks to fill potentially insolvent third party protocols with funds and honor their own (insolvent) claims into these protocols. Example: Alice has 100 shares on a specific AAVE market but this market is insolvent / has bad debt This market is the primary market of VaultV2 Alice deposits 100 assets into this market which is auto allocated Alice withdraws her (insolvent) claim as it now became solvent Alice can withdraw her 100 assets directly if there are idle assets in VaultV2 or Alice can wait until a user deposits and temporarily trigger the supply limit to be hit to deallocate the users deposit back and then withdraw it.
Recommendations	Consider keeping this scenario in mind. There is no simple fix.
Comments / Resolution	Acknowledged

Issue_08	Truncation when interacting with third-party protocols can result extended side-effects stemming from inconsistency between adapterAllocated and nominal external value
Severity	Low
Description	VaultV2 tracks adapter exposure through <code>adapterAllocated[adapter]</code> and <code>adaptersAllocated</code>, and adapter contracts track their own global exposure through <code>globalAllocated[collateral]</code>. Core vault logic, including <code>instantWithdraw()</code>, relies on these values rather than on the adapter's true remaining economic position. In the Aave integration, the adapter's real external assets are measured from its AToken-backed position, while the vault-side allocation values are updated only by the nominal amount returned during deallocation. If the external adapter position returns slightly less than the nominal principal recorded in storage (due to ie. rounding against the adapter when interacting with AAVE/Morpho), and a withdrawal/deallocation flow is still completed using separate idle vault liquidity, the vault can be left with a stale positive adapter allocation even though the real adapter position has already gone to zero. The issue is that VaultV2 has no explicit write-down path when an adapter interaction returns less than the nominal amount recorded in <code>adapterAllocated</code> / <code>adaptersAllocated</code>. In the Aave path, this can happen due to external scaled-balance / liquidity-index rounding, so that a full economic round-trip leaves the adapter with zero real position while the vault still records a positive dust amount. Simple example: - Initial values: - <code>vaultV2.adapterAllocated[adapter] = 0</code> - <code>vaultV2.adaptersAllocated = 0</code>

```

- adapter.globalAllocated[collateral] = 0
- totalCollateralShares[collateral] = 0
- vaultShares[collateral][vault] = 0
- aTokenAdapterBalance = 0

- AaveLiquidityIndex = 1.165904213550150726284100961e27 (ray
precision)

    2. Vault allocates 100e18 assets to Aave adapter:
// Vault interaction
- vaultV2.adapterAllocated[adapter] = 100e18
- vaultV2.adaptersAllocated = 100e18

// Adapter interaction
- adapter.globalAllocated[collateral] = 100e18

// Aave interaction
- scaledMintedATokens = rayDivFloor(100e18, liquidityIndex) =
85.770339310724650014
- aTokenAdapterBalance = rayMulFloor(scaledMintedTokens,
liquidityIndex) = 99.999999999999999999e18 --> rounded down 1
wei

// continue with adapter interaction
/// Preview deposit (round down)
- mintedShares = fullMulDiv(100e18, [totalCollateralShares + 1],
[aTokenAdapterBalance + 1] = 100e18
- vaultShares[collateral][vault] = 100e18
- totalCollateralShares[collateral] = 100e18
// End tx

    3. Now, for simplicity, let's assume that no yield has accrued
    (liquidityIndex remains unchanged) and that everything is
    deallocated (100e18).
// Adapter interaction
/// Preview redeem (round down)
- getVaultAssets = fullMulDiv(100e18,
[99.999999999999999999e18+1], [100e18+1]) =

```

```

99.999999999999999999e18
- deallocatable = min(collateral.balanceOf[adapter] +
min(getVaultAssets, collateral.balanceOf[aToken]),
adapterAllocated[adapter])
- deallocatable = min(0 + 99.999999999999999999e18, 100e18)
- deallocated = min(99.999999999999999999e18, 100e18)
- deallocated = 99.999999999999999999e18
- amountToWithdraw = 99.999999999999999999e18

// aave withdraw works in a way that it should withdraw exactly the
assets specified
amountWithdrawn = 99.999999999999999999e18

// continue adapter interaction
/// previewWithdraw (round up)
burnedShares = fullMulDivUp(9999999999999999999, 100e18,
9999999999999999999) = 100e18
- vaultShares[collateral][vault] = 0
- totalCollateralShares[collateral] = 0

- globalAllocated[collateral] = 100e18 - deallocated = 1 wei

// final interaction with vault
- adapterAllocated[adapter] = 100e18 - deallocated = 1 wei
- adaptersAllocated = 100e18 - deallocated = 1 wei
// end tx

```

4. So we end up with the following state, and nothing is allocated

```

- vaultV2.adapterAllocated[adapter] = 1
- vaultV2.adaptersAllocated = 1
- vaultV2.totalStake = 100e18
- vaultV2.collateralBalance = 99.999999999999999999e18

- adapter.globalAllocated[collateral] = 1
- totalCollateralShares[collateral] = 0
- vaultShares[collateral][vault] = 0

```

	If a user exit is absorbed using separate idle liquidity (i.e. 1 wei of idle liquidity to withdraw the full 100e18) and therefore completes successfully, the stale adapter residue remains in storage, while <code>totalStake</code> decreases to zero. The vault then continues to treat that dust as real adapter exposure. This directly affects later logic such as <code>instantWithdraw()</code>, which checks whether the remaining <code>adaptersAllocated</code> is within <code>_maxAllocatable()</code>, and adapter management actions like limit reduction or removal, which gate on <code>adapterAllocated[adapter]</code>. The root cause is that the vault updates allocation storage from the nominal deallocated amount but never reconciles the result against the adapter's actual remaining economic position. Impact 1: <code>instantWithdraw()</code> can revert on the final safety check even when the vault already holds enough idle collateral to satisfy the user, because stale positive <code>adaptersAllocated</code> makes the vault believe adapter exposure still exists Impact 2: adapter removal or limit reduction to zero can become impossible because the vault still records positive <code>adapterAllocated[adapter]</code> for an economically dead position Impact 3: future adapter headroom is unnecessarily reduced because <code>adaptersAllocated</code> and <code>globalAllocated</code> remain overstated Impact 4: fresh deposits can silently recapitalize the old accounting hole rather than entering a clean state, so new users can end up backfilling historical adapter-side losses or rounding deficits Impact 5: repeated successful cycles can accumulate stale residue over time, but only when each cycle's shortfall is subsidized by separate idle vault liquidity, making the issue operationally intermittent and therefore harder to detect.
Recommendations	Consider elaborating whether the above impacts can become dangerous, as they will impact the <code>VaultV2</code> contract which is not within this scope.

Comments / Resolution	Acknowledged. The inconsistency still occurs, but the team introduced a <code>recover()</code> function that clears phantom allocations when new funds are added.
-----------------------	---

Issue_09	Precision loss concerns
Severity	Low
Description	In multiple parts of the contract, rounding may cause small amounts of wei dust to become locked. For low-decimal or high-value tokens, this accumulation could result in a meaningful loss over the protocol's lifetime.
Recommendations	Consider which tokens are expected to be used
Comments / Resolution	Acknowledged

Issue_10	Missing <code>_gap</code> implementation for inherited base
Severity	Low
Description	The <code>Adapter</code> contract is meant to be inherited and at the same time serves as an implementation contract for a proxy. The absence of having a <code>_gap</code> variable introduces risk of storage collision if an upgrade is executed which includes new storage slots within the <code>Adapter</code> contract.
Recommendations	Consider implementing the <code>_gap</code> variable.
Comments / Resolution	Fixed

Issue_11	Supply limits within AAVE and Morpho are not considered
Severity	Low
Description	Please note that this issue is technically within the MorphoVaultV2Adapter and AAVEV3Adapter contracts but since the description is the same, it will be mentioned within this section. Whenever an allocation is being executed, funds will be deposited in third-party protocols. The only allocation constraint is the global limit and there is no such check which searches for individual supply limits in the third party protocols. If such a deposit reverts, whereas it could partially allocate, funds are simply skimmed back to the VaultV2 contract and remain idle there without any yield accrual and are required to be allocated manually.
Recommendations	We do not recommend the incorporation of supply limits because the impact of this issue is very limited and such an incorporation may introduce other vulnerabilities or side-effects. Consider regularly inspecting the VaultV2 for idle assets.
Comments / Resolution	Acknowledged.

Issue_12	Virtual share mechanism can result in side-effects
Severity	Low
Description	Please note that this issue is technically within the MorphoVaultV2Adapter and AAVEV3Adapter contracts but since the description is the same, it will be mentioned within this section. Both adapters pool all Symbiotic vaults sharing the same collateral into one internal ownership accounting. It tracks pooled ownership with <code>totalCollateralShares[collateral]</code> and <code>vaultShares[collateral][vault]</code>, while the real external position is measured from the adapter's Aave aToken balance (or Morpho share token amount). Each vault's current claim on that pooled Aave position is computed with <code>ERC4626Math.previewRedeem(...)</code>, and both <code>skim()</code> and <code>deallocate()</code> burn internal shares with <code>ERC4626Math.previewWithdraw(...)</code>. The conversion library uses ERC4626 virtual offsets by adding +1 to both total shares and total assets. The issue is that the ERC4626 virtual-offset math makes the internal share accounting non-exhaustive with respect to the real Aave (Morpho) position. When the adapter's real Aave asset balance increases relative to the internal share supply, the total claim of all real shares can become strictly less than the actual aToken-backed assets. In that state, <code>skim()</code> or <code>deallocate()</code> can burn all real internal shares while the adapter still retains a positive Aave position. A reachable path is to allocate a very small amount into an empty Aave adapter so the adapter has only a very small real share supply, then increase the adapter's aToken balance without minting internal shares, for example by direct aToken donation. Once <code>_getVaultAssets(vault)</code> rises above <code>adapterAllocated</code>, <code>skim()</code> can withdraw a positive amount and burn the last real internal share, leaving <code>totalCollateralShares == 0</code> while aToken balance remains positive. The root cause is the use of ERC4626 virtual-offset rounding as an internal pooled ownership system for adapter accounting: the formulas floor vault claims, ceil share burns, and do

not guarantee that the set of all real shares exactly covers the real external Aave assets.

Impact 1: real Aave assets can become stranded inside the adapter because no vault retains internal shares to claim them through `_getVaultAssets`, `skimmable`, or `deallocatable`

Impact 2: the adapter can simultaneously retain stale positive `adapterAllocate`, which will have second order effects into `instantWithdraw` and adapter removal.

Example:

Initial state after the first tiny allocation:

- `adapterAllocated` = 1
- `globalAllocated` = 1
- `totalCollateralShares` = 1
- `vaultShares` = 1
- `aToken` balance = 1

Someone donates 2 `aToken` units to the adapter, or equivalent yield accrues:

- `aToken` balance = 3

Then:

- `_getVaultAssets` = `previewRedeem(1, 3, 1)` = $\text{floor}[(1 * (3 + 1)) / (1 + 1)] = 2$
- `skimmable` = $2 - 1 = 1$

`skim()` withdraws 1 and burns:

- `previewWithdraw(1, 1, 3)` = $\text{ceil}[(1 * (1 + 1)) / (3 + 1)] = 1$

Final state:

	- totalCollateralShares = 0 - vaultShares = 0 - aToken balance = 2 - adapterAllocated = 1 - globalAllocated = 1 At this point, the adapter still holds real Aave assets, but no vault has internal shares left to claim them. This is an invalid state with second order effects that will flow into the VaultV2 scope
Recommendations	Consider further evaluating side-effects from this, as these will likely impact the VaultV2 contract which is out of scope for this audit.
Comments / Resolution	Fixed by removing the share mechanism.

Issue_13	External incentives cannot be claimed and could generate additional issues
Severity	Low
Description	When interacting with Morpho or Aave, for some tokens, the holder of the third-party protocol shares (for example, aTokens or Morpho vault shares) may become eligible for rewards created to incentivize specific markets or vaults. See the following for more details: - https://aave.com/docs/aave-v3/concepts/incentives - https://docs.morpho.org/learn/concepts/rewards/ The adapters do not include any functionality to handle these types of external rewards. The main concern is that some of these rewards can be claimed on behalf of the adapter by anyone, and the claimed rewards will be sent automatically to the adapter contract. If the transferred token does not match any of the collateral tokens managed by the contract, those tokens may become permanently locked in the contract. On the other hand, if the token does match one of the managed collateral tokens, it could be extracted as yield. This also introduces additional issues, similar to those caused by a donation of that collateral token, as explained in other notes in this report. It can also result in inflation-attack style issues. For example, when the reward token == collateral token is sent to the contract, the first vault to deallocate funds for that collateral token could capture the entire reward amount as yield and redirect it to its own vault. Those rewards, once converted into yield, are not shared proportionally. This issue is discussed separately. In addition, these rewards, once effectively “donated” to the contract, could become part of other attack paths that allow a profit to be extracted.
Recommendations	Consider accounting for the external incentives of these third-party protocols and handling them appropriately.

Comments / Resolution	Acknowledged. Third-party protocol positions are now owned by separate proxies pointing to the same implementation. The current implementation does not include functionality to extract external incentives, but it can be upgraded to enable this if needed. The upgrade needs care, as it involves upgrading the implementation of multiple proxies at the same time.
------------------------------	---

Issue_14	Deposits revert when the underlying protocol's withdraw fails inside <code>skim</code>
Severity	Low
Description	Both adapters, when allocating, start with: <pre><i>_skim(msg.sender); if (skimmable(msg.sender) > 0) revert SkimFailed();</i></pre> Inside <code>_skim</code>, the underlying <code>Pool.withdraw/morphoVault.withdraw</code> is wrapped in a <code>try/catch</code>, so a revert there is silently swallowed. However, <code>skimmable</code> remains positive afterwards, which trips the <code>SkimFailed</code> guard and reverts the entire allocate call, including the call made from <code>VaultV2.deposit</code> against <code>adapters[0]</code>. The result is that deposits to the vault are DoSed for as long as the underlying withdraw cannot be served. The conditions that make the underlying withdraw revert are reachable in normal market operation: - Aave V3 → reserve over-utilized, reserve inactive, or reserve paused. - Morpho VaultV2 → aggregate resolvable liquidity (idle + cascade through liquidity adapters) below the skim amount, vault paused, or any of the curator-configured gates reverting/returning false for the proxy or the adapter. The DoS window can be triggered by any actor who already has a supplied position in the same Aave reserve or the same Morpho vault: withdrawing their own funds shrinks available liquidity below <code>skimmable[vault]</code> and frontrunning the vault's next deposit forces the revert. The attack surface is bounded by the attacker's existing capital, but it is essentially free relative to that capital and self-resolves only when liquidity returns or the gate is removed. We acknowledge that this change was introduced to prevent depositors from sandwiching yield by forcing <code>skim</code> to revert.

	However, this does not fully address the issue. An attacker can still force <code>skim</code> to revert in subsequent adapters within the vault, and as long as the first adapter's skim does not revert, the overall deposit may still succeed and sandwich the yield accrued by the other adapters. This represents a separate issue.
Recommendations	The design of reverting the deposit if the skim fails may be considered a design choice, but it could cause serious DoS under stress.
Comments / Resolution	Acknowledged.

Issue_15	Third-party protocols can lock funds due to different settings
Severity	Informational
Description	Please note that this issue is technically within the MorphoVaultV2Adapter and AAVEV3Adapter contracts but since the description is the same, it will be mentioned within this section. When depositing funds into third-party protocols, this inherently exposes these funds to configuration settings within the underlying protocols or pausing mechanisms, which can result in these funds becoming fully locked.
Recommendations	Consider keeping this in mind.
Comments / Resolution	Acknowledged. Phantom or stranded allocations can be recovered via the <code>recover()</code> function.

Issue_16	Setting 100% protocol/curator fee could break the main flows of vault V2
Severity	Informational
Description	Please note that this issue technically belongs to the Vault V2/DonationRewards contracts. However, since the relevant flow passes through the adapter contracts, we believe it is still worth mentioning here. Setting the protocol/curator share to 100% could make sense if they decide that, for a period of time, all yield should be captured for protocol or curator purposes. In that case, no yield would be redirected to the vault, but the donationRewards contract would still call <code>vaultV2::donate[0]</code>, and donate would revert when passed a zero amount. Given that deposit/withdraw allocate/deallocate operations skim adapters first, if any yield is realized, the entire transaction would revert.
Recommendations	Consider taking this into account.
Comments / Resolution	Acknowledged.

Issue_17	Donation rewards are not automatically allocated
Severity	Informational
Description	Please note that this issue technically belongs to the Vault V2 contract. However, since the relevant flow passes through the adapter contracts, we believe it is still worth mentioning here. Whenever the donate function is triggered, it will increase activeStakes but these funds remain idle in the VaultV2 until they get manually allocated, hence no yield is produced.
Recommendations	Consider keeping this in mind.
Comments / Resolution	Acknowledged.

Issue_18	Potential side-effects from unexpected <code>getNoAdaptersSize</code> change
Severity	Informational
Description	Please note that this issue technically belongs to the Vault V2 contract. However, since the relevant flow passes through the adapter contracts, we believe it is still worth mentioning here. Within <code>VaultV2</code>, the <code>getNoAdaptersSize</code> return value from the <code>UniversalDelegator</code> is used as follow: <pre>function _maxAllocatable() internal view returns (uint256) { return totalStake().saturatingSub(UniversalDelegator(delegator).getNoAdaptersSize()); }</pre> If for example the size is decreased and a withdrawal is executed, the withdrawal decreases <code>totalStake</code> which then in turn determines <code>_adaptersOwe</code>. The problem is that now the <code>getNoAdaptersSize</code> is decreased which increases <code>maxAllocatable</code> and if that is increased, it will decrease <code>_adaptersOwe</code> which results in insufficient deallocation from the adapter to honor the withdrawal. Since this issue is technically out of scope, we have simply marked it as informational.
Recommendations	We recommend carefully evaluating whether such a state can happen, as this depends on the underlying logic within the <code>UniversalDelegator</code> contract. It should also be considered what issues may arise if this value is increased.
Comments / Resolution	Acknowledged.

Issue_19	Skim is not capped with deallocatable
Severity	Informational
Description	The <code>skim</code> function is currently not capped with the deallocatable amount (unlike deallocations), which means in case a partial amount can be skimmed, it will not happen.
Recommendations	We do not recommend a change because this does not expose a real issue and there could be unexpected side-effects from partial skims even if those are not obvious at this point in time.
Comments / Resolution	Acknowledged.

Issue_20	Yield from slashes amount technically belongs to slasher
Severity	Informational
Description	Please note that this issue technically belongs to the Vault V2 contract. However, since the relevant flow passes through the adapter contracts, we believe it is still worth mentioning here. Whenever the VaultV2 is being slashed while there are insufficient funds in the adapter to honor the withdrawal, it will just attempt to claim these funds at a later point via the sync logic. However, until that point in time they will still sit in the adapter, actively earning yield, which then should belong to the slasher.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_21	Truncation mechanisms will benefit other existing vaults in multi-vault scheme
Severity	Informational
Description	Please note that this issue is technically within the MorphoVaultV2Adapter and AAVEV3Adapter contracts but since the description is the same, it will be mentioned within this section. Both adapters are compatible with multiple vaults (e.g. Symbiotic Vault A and B can deposit into the same AAVE market) and for this purpose a share accounting system is used (see Appendix). The share accounting system always rounds against the favor of the executor which will then result in a favor for the other existing vaults. This is usually only a dust concern but it is important to be pointed out. Likewise, skimming will round up burned shares which can decrease principal: Two vaults share the same Morpho pool: - Vault A allocated 50 - Vault B allocated 50 - external assets rise from 100 to 110 After sequential skims using actual integer math: - both vaults can end with adapterAllocated = 50 - but each only claims 49 So the 10 units of yield are harvested, but 2 units of principal-like claim are lost from the vault claims.
Recommendations	Consider taking this into account.
Comments / Resolution	Fixed by removing the share mechanism. Each vault now directly owns its shares in its third-party protocol.

Issue_22	Gas grieving via dust donations to the proxy account
Severity	Informational
Description	Both adapters, when allocating and deallocating, invoke <code>_skim[msg.sender]</code> at the start of every call. Inside <code>_skim</code>, any non-zero <code>skimmable[vault]</code> triggers the full yield round trip: A withdrawal from the underlying protocol, a conditional <code>safeApproveWithRetry</code> to the rewards contract, and an <code>IRewards.distributeDonationRewards(vault, amount)</code> call, which itself re-enters the vault via <code>donate()</code>. An external attacker can transfer a few wei of aToken/Morpho shares directly to the deterministic proxy account at any time. The next caller that interacts with the adapter – typically a curator or the vault itself during a deposit/allocate flow – then pays the gas for that entire round trip, even when the economic value of the skim is dust. The donor pays only the cost of a single ERC20 transfer; the victim pays the cost of an Aave/Morpho withdrawal plus a callback into the vault and the rewards bookkeeping. This is a pure grieving vector: no funds are stolen, and the donated dust is genuinely distributed as yield, but the gas cost is inflated for legitimate users at very low cost to the attacker.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_23	<code>Adapter.recover()</code> is incompatible with fee-on-transfer collateral
Severity	Informational
Description	<code>Adapter.recover()</code> pulls collateral from the vault using a fixed amount: <pre><i>IVaultV2{vault}.collateral().safeTransferFrom(msg.sender, address(this), amount);</i> _recover(vault, amount); skim(vault);</pre> The same amount is then passed to <code>_recover</code> → <code>deallocateAdapter</code>, and the subsequent skim assumes the adapter actually received amount units. For FoT token, the real balance delta on the adapter is amount - fee, so all downstream accounting (<code>adapterAllocated</code>, <code>globalAllocated</code>, <code>skimmable</code>) drifts by the fee. Depending on the path, this either reverts (insufficient balance to forward the full amount to the underlying protocol) or silently miscredits the vault. Today, the in-scope adapters target Aave V3 and Morpho VaultV2, neither of which is configured against FoT collateral, so the issue is latent. However, the <code>Adapter</code> base contract is designed to be extended, and any future adapter that ever needs to support a FoT-style asset will inherit this broken recovery path.
Recommendations	Consider keeping that in mind.
Comments / Resolution	Acknowledged.

MorphoVaultV2Adapter

The `MorphoVaultV2Adapter` contract is inheriting the `Adapter` contract and serves as a direct adapter for the `VaultV2` contract. Whenever a deposit or a manual allocation is happening, this contract will receive the corresponding amount of tokens and allocates these to the MorphoVault ([VaultV2 contract](#)). From there on, it will then be either allocated to a [Morpho](#) market or further allocated to the [MetaMorpho](#) vault, which are investment strategies to accrue yield on the underlying tokens. While abstracting away the specific control-flows, in a nutshell, is simply a deposit into a third party protocol where funds can be withdrawn again [under regular operations].

Moving forward, the adapter will accrue yield which is redistributed via the `VaultV2.donate` mechanism.

Appendix: Share Accounting

The `MorphoVaultV2Adapter` maintains its own internal two-layer share system to track each Symbiotic vault's proportional claim on the adapter's pooled Morpho position. When multiple Symbiotic vaults share the same adapter, they all deposit into a single Morpho vault position represented by `morphoVault.balanceOf(adapter)`.

The adapter subdivides this single position using `vaultShares[morphoVault][vault]` (per-vault) and `totalVaultShares[morphoVault]` (aggregate), which are maintained independently from Morpho's own share accounting.

On allocation, the adapter computes `mintedShares` via `ERC4626Math.previewDeposit(amount, totalVaultShares, _getAdapterAssets)` using the formula $\text{floor}[\text{amount} \times (\text{totalVaultShares} + 1) / (_getAdapterAssets + 1)]$, where `_getAdapterAssets` is itself a live read from `morphoVault.previewRedeem(morphoVault.balanceOf(adapter))`.

On deallocation, the adapter computes `burnedShares` via `ERC4626Math.previewWithdraw(amountToWithdraw, totalVaultShares, _getAdapterAssets)` using $\text{ceil}[\text{amountToWithdraw} \times (\text{totalVaultShares} + 1) / (_getAdapterAssets + 1)]$

Appendix: Skim Mechanism

The skim mechanism is the sole path for extracting yield from the adapter's Morpho position and crediting it to the Symbiotic vault's stakers. It operates by detecting the surplus between the adapter's share-weighted claim (`_getVaultAssets`) and the vault's tracked allocation (`adapterAllocated`), withdrawing that surplus from Morpho, and distributing it via the Rewards contract as a donation to the vault's activeStake. Yield detection begins with `skimmable[vault]`, computed as `_getVaultAssets[vault].saturatingSub(IVaultV2[vault].adapterAllocated[address[this]])`.

When the Morpho position earns yield (from Morpho Blue market interest), `_getAdapterAssets` increases (Morpho's `previewRedeem` returns a higher value for the same number of Morpho shares), which flows through to `_getVaultAssets` increasing above the vault's `adapterAllocated`.

Privileged Functions

- `setMorphoVault`

Core Invariants:

INV 1: Only the Vault can call `allocate` and `deallocate`

INV 2: An active `MorphoVault` tied to a `VaultV2` cannot be changed

INV 3: `vaultShares[morphoVault][VaultV2]` must be increased by the minted share amount

INV 4: Each `MorphoVault` can be tied to multiple Symbiotic Vaults

INV 5: `_getVaultAssets` must return the token amount which is entitled to the `VaultV2` based on its shares in context to the overall shares

INV 6: `_getAdapterAssets` must return all underlying assets for this adapter

Issue_24	Exchange rate decrease within Morpho is not accounted for
Severity	High
Description	VaultV2 treats adapter exposure as principal-like value through adapterAllocated[adapter] and adaptersAllocated, while user share pricing is based on activeStake and activeShares. In the Morpho path, the adapter separately tracks the vault's entitlement to the external Morpho vault through internal adapter shares ($\text{vaultShares} / \text{totalVaultShares}$) and derives the vault's current economic claim via _getVaultAssets(vault). The external Morpho position can lose value over time because of realized losses, rounding losses, or external management/performance fee effects. However, those decreases are not automatically propagated into VaultV2.activeStake or VaultV2.adapterAllocated, so the vault can continue to price user exits as if the old principal were still fully present. The issue is that VaultV2 assumes the nominal amount recorded as allocated to the Morpho adapter is still economically recoverable, while the adapter itself settles deallocation against the current reduced external claim. If the external Morpho share price has decreased, _getVaultAssets(vault) becomes smaller, but activeStake and adapterAllocated remain stale-high. A user can then withdraw based on the stale vault share price and receive a claim larger than their fair economic share. When the vault later recalls funds from the adapter to honor that exit, the adapter burns its internal shares against the reduced external asset base. In the extreme case, the adapter can burn the vault's full adapter-share position to return only the reduced current claim, while the vault still records a large positive activeStake and positive adapterAllocated afterward:

Alice and Bob each deposit 50 into the same VaultV2:

- activeStake = 100
- activeShares = 100
- adapterAllocated = 100
- adapter internal shares for the vault = 100
- external Morpho adapter assets = 100

Now the external Morpho share price falls by 50%, so:

- external adapter assets = 50

but VaultV2 still keeps:

- activeStake = 100
- adapterAllocated = 100

Alice calls withdraw[50].

Because VaultV2 still uses activeStake = 100, Alice receives a withdrawal claim for 50, even though her fair economic share after the external loss is only 25.

When Alice later claims, the vault recalls 50 from the adapter.

The adapter's current claim for the vault is only 50, so:

- amountToWithdraw = 50
- burnedShares = previewWithdraw(50, 100, 50) = $\text{ceil}(50 * 101 / 51) = 100$

So the adapter burns all of the vault's adapter shares to return that reduced current claim of 50. Alice is paid 50.

After that:

- activeStake on VaultV2 is still 50
- activeShares for Bob are still 50

but the external adapter position is already exhausted and adapterAllocated is still 50

The root cause is the absence of any mark-to-market or loss-realization mechanism that synchronizes VaultV2's principal accounting with the adapter's current external value before user exits are priced.

There are multiple different impacts:

Impact 1: early exiters can receive more than their fair economic share when the external Morpho position has already lost value, which externalizes losses onto remaining vault users

Impact 2: remaining users can be left holding active vault shares that are economically unbacked even though VaultV2 storage still reports positive activeStake

Impact 3: the vault can enter a hidden-insolvency state where the external adapter position is exhausted or largely depleted, but adapterAllocated, adaptersAllocated, and globalAllocated still remain positive

Impact 4: later instantWithdraw() can revert because stale positive adapter allocation causes the _maxAllocatable() safety check to fail even though the real external position is gone

Impact 5: future deposits can silently recapitalize historical Morpho losses because new capital mints against stale vault accounting and backfills the old hole

Impact 6: external Morpho management-fee accrual or fee

	increases can trigger the issue even in the absence of explicit market losses, because those fees reduce external share value without any matching Symbiotic-side write-down Impact 7: slash accounting can also be distorted, because the vault may appear to have more adapter-backed value than is economically recoverable, which can convert supposed immediate slash value into hidden owed slash or underpayment.
Recommendations	Consider implementing either a solution that accounts for this behavior or prevent deallocations when the current vault assets are less than what has been allocated, which however could be often a natural state post-skimming because the required shares are rounded up and as such this might produce such a state where it can naturally happen that <code>adapterAllocated > _getVaultAssets</code> and as such prevents deallocations while the exchange rate has not decreased. This should be carefully tested with fuzzing to make sure it does not “false-revert”. Potential edge-cases might occur with either solution and should be analyzed carefully.
Comments / Resolution	Partially resolved. Any type of loss can be recovered via <code>Adapter::recover()</code> function and in the worst case the <code>forceDeallocation</code> function can be used. However, in this scenario, there is still a FCFS situation within <code>VaultV2</code> after force deallocation. The main FCFS scenario has been fixed with the buffer logic.

Issue_25	Rounding-only deposits can wedge <code>DEALLOCATE_BUFFER</code> and DoS normal deallocations
Severity	Medium
Description	The underwater guard inside <code>MorphoVaultV2Adapter._deallocate</code> is gated on: <pre>adapterAllocated.saturatingSub(getAssets) > DEALLOCATE_BUFFER [= 10].</pre> The intent is to block deallocations only when the position has experienced a real loss above 10 wei. However, the same condition is reachable through Morpho's standard mint-side rounding, with no loss event in the Morpho vault. A permissionless attacker (or any user, organically) can chain micro-deposits whose individual rounding loss is exactly 1 wei but which never get reabsorbed by share appreciation. Example: - 18 decimal collateral (<code>virtualShares = 1</code>). - <code>TotalAssets</code> = 10000 - <code>TotalSupply</code> = 9999 - <code>adapterAllocated</code> = 100 - <code>ProxyMorphoShares</code> = 100 Attacker calls <code>VaultV2.deposit[2]</code> eleven times. Each call mints: - $\text{floor}[2 \cdot (\text{totalSupply}+1)/(\text{totalAssets}+1)] = 1$ share for 2 wei 1. $\text{floor}[2 \cdot 10000 / 10001] = 1$ --> Diff 1 2. $\text{floor}[2 \cdot 10001 / 10003] = 1$ --> Diff 2 3. $\text{floor}[2 \cdot 10002 / 10005] = 1$ --> Diff 3 4. $\text{floor}[2 \cdot 10003 / 10007] = 1$ --> Diff 4 5. $\text{floor}[2 \cdot 10004 / 10009] = 1$ --> Diff 5 6. $\text{floor}[2 \cdot 10005 / 10011] = 1$ --> Diff 6 7. $\text{floor}[2 \cdot 10006 / 10013] = 1$ --> Diff 7 8. $\text{floor}[2 \cdot 10007 / 10015] = 1$ --> Diff 8 9. $\text{floor}[2 \cdot 10008 / 10017] = 1$ --> Diff 9

	10. $\text{floor}[2 * 10009 / 10019] = 1 \rightarrow \text{Diff } 10$ 11. $\text{floor}[2 * 10010 / 10021] = 1 \rightarrow \text{Diff } 11$ (Trigger the underwater guard) We end up: - <code>totalAssets</code> = 10023 - <code>totalSupply</code> = 10011 - <code>ProxyMorphoShares</code> = 111 - <code>adapterAllocated</code> = 122 - <code>getAssets</code> = <code>previewRedeem</code>(111) = $\text{floor}[111 \cdot 10023 / 10011] = 111$ - $\text{diff} = 122 - 111 = 11 > 10$ - so <code>_deallocate</code> short-circuits to 0 from this point on. The wedge has these consequences until either the curator calls <code>forceDeallocate</code> or organic yield lifts <code>getAssets</code> enough to bring $\text{diff} \leq 10$: - Any role-holder calling <code>deallocateAdapter</code> against this adapter receives 0 back. - <code>VaultV2.claim</code>, <code>instantWithdrawal</code>, and <code>syncOwedSlash</code> all rely on <code>deallocateAdapters[]</code>, so the fan-out of user withdrawals can come up short and revert with <code>InsufficientAmount</code> if this adapter is the meaningful funded one. - No loss has occurred in Morpho. The buffer fired purely on rounding. The trigger is permissionless and cheap (11 deposits of 2 wei plus gas). It should be noted that this can happen organically as well, through normal deposits.
Recommendations	Consider implementing a dynamic buffer which is settable by governance.
Comments / Resolution	Partially resolved, the buffer has been increased. It is still theoretically possible but now requires more executions.

Issue_26	Minimal allocations concerns
Severity	Low
Description	In <code>MorphoVaultV2Adapter.allocate()</code>, the adapter's internal <code>mintedShares</code> are computed before the external <code>deposit()</code> call is made to Morpho: <pre>uint256 mintedShares = ERC4626Math.previewDeposit(amount, totalVaultShares[morphoVault], _getAdapterAssets(morphoVault)); try IMorphoVaultV2(morphoVault).deposit(amount, address(this)) returns (uint256) { vaultShares[morphoVault][msg.sender] += mintedShares; totalVaultShares[morphoVault] += mintedShares; } catch { ... }</pre> Morpho Vault V2's <code>previewDeposit</code> uses floor division (<code>mulDivDown</code>): <pre>shares = assets.mulDivDown(totalSupply + virtualShares, totalAssets + 1)</pre> In a mature vault where yield has accrued and <code>totalAssets > totalSupply</code>, a small deposit can result in 0 Morpho shares being minted. Crucially, Morpho's <code>createShares(onBehalf, 0)</code> does not revert, it silently succeeds. The try block succeeds, and the adapter credits <code>mintedShares</code> (computed from its own internal formula) even though the adapter received 0 actual Morpho shares. Example with WETH: Initial Morpho vault state: - Token: <code>WETH</code> (18 decimals, <code>virtualShares = 1</code>) - <code>morphoVault.totalSupply = 100e18</code> - <code>morphoVault.totalAssets = 200e18</code> --- Step 1: Alice deposits 1 wei: <pre>// VaultV2 - mintedShares = previewDeposit(1 x [0+1] / [0+1] = 1) - adapterAllocated[adapter] = 1 - adaptersAllocated = 1 - totalStake[] = 1 // Adapter allocate(1):</pre>

- `mintedShares = previewDeposit(1, totalVaultShares, _getAdapterAssets) = 1 x (0+1) / (0+1) = 1`
- `globalAllocated = 1`

// Morpho deposit(1, adapter):

- `morpho.previewDeposit(1) = 1 x (100e18 + 1) / (200e18 + 1) = 0`

// Adapter state after:

- `vaultShares[morphoVault][vault] = 1`
- `totalVaultShares[morphoVault] = 1`
- `morphoVault.balanceOf(adapter) = 0`
- `_getAdapterAssets() = 0`
- `VaultV2 collateral balance = 0`

--- Step 2: Alice requests a withdrawal

- Active shares burned, pending withdrawal queued for 1 wei
- `totalStake() = 1` (pending withdrawal still counts toward slashable stake)
- `adaptersAllocated = 1` (unchanged)

--- Step 3: Alice tries to claim

// `claim() -> deallocateAdapters()`:

- `_adaptersOwe() = adaptersAllocated.saturatingSub(_maxAllocatable()) = 1 - 1 = 0` -> loop skips, nothing deallocated
- `_safeTransferOut(alice, 1) -> REVERT`

--- Step 4: Someone donates 1 wei of WETH directly to VaultV2

VaultV2 collateral balance = 1 (donated, not tracked by `totalStake` or active accounting)

--- Step 5: Alice claims again

// `claim() -> deallocateAdapters()`:

- `_adaptersOwe() = 1 - 1 = 0` -> does nothing

// VaultV2 pays Alice using the donated balance:

- `_safeTransferOut(alice, 1) -> succeeds`

	<pre>// Alice's pending withdrawal settled, her stake removed from delegator: - totalStake[] = 0 - adaptersAllocated = 1 <- never decremented, phantom allocation - _maxAllocatable[] = 0 --- Vulnerable final state --- - totalStake[] = 0 - adaptersAllocated = 1 - _maxAllocatable[] = 0 --- Step 6: Bob deposits 100e18 WETH, allocator allocates to adapter - totalStake[] = 100e18 - adaptersAllocated = 1 + 100e18 <- phantom 1 wei still counted - _maxAllocatable[] = 100e18 // Bob tries instantWithdraw: // deallocateAdapters[] runs first, correctly recovers Bob's legitimate portion. // Then the check: _maxAllocatable[] < adaptersAllocated -> 100e18 < 100e18 + 1 -> TRUE -> revert InsufficientAmount[] instantWithdraw is DoS'd for all users of this vault. Until 1 wei is donated to the adapter, and accounting could be restored once the wei is extracted from it.</pre>
Recommendations	Consider introducing a minimum investment amount, we also refer to the general truncation scenario which can cause such a state.
Comments / Resolution	Partially resolved by introducing a recover[] function, where it can reset the dust phantom allocations to zero. It has to be noted that this issue still theoretically exists.

Issue_27	A new Morpho vault cannot be set when the share price is degraded and the proxy holds dust shares..
Severity	Low
Description	The function <code>setMorphoVault</code> prevents switching vaults if the proxy holds any shares at all: <pre> if (curMorphoVault != address[0] && IMorphoVaultV2[curMorphoVault].balanceOf(getAccount[vault]) > 0) { revert ActivePosition(); } </pre> Because the check relies on the raw share balance, it is vulnerable to manipulation. Any external party can transfer a negligible amount of shares (e.g., 1 wei) to the proxy address. Once this happens, the proxy is considered to have an “active position,” and the vault migration is blocked. Whether this situation can be resolved depends on whether those shares are redeemable. In certain conditions, specifically when the vault’s share price falls below 1, small amounts of shares may round down to zero when redeemed. In such cases: - The shares cannot be skimmed. - They cannot be removed through deallocation mechanisms, including forceful ones. As a result, the proxy remains stuck with a nonzero share balance that has no redeemable value, yet still prevents migration. Example: Healthy price (price ≥ 1) → donation is auto-cleaned. Morpho state <code>assets</code> = 1001, <code>supply</code> = 1000. Attacker transfers 1 share → <code>morpho.balanceOf(proxy)</code> = 1. - <code>previewRedeem</code>(1) = floor(1002 / 1001) = 1

	- <code>skimmable</code> = 1. Next skim withdraws 1 wei (burns 1 share), proxy back to 0. - <code>setMorphoVault</code> works on the next call. Degraded price ($\text{price} < 1$) → donation creates a vault migration block. Morpho state <code>assets</code> = 1000, <code>supply</code> = 1001. Attacker transfers 1 share → <code>morpho.balanceOf(proxy)</code> = 1.. - <code>previewRedeem(1)</code> = $\text{floor}(1001 / 1002) = 0$ - <code>skimmable</code> = 0, so skim is a no-op. - The donation is not in <code>adapterAllocated</code>, so <code>deallocatable</code> = 0 and <code>_deallocate</code> (including via <code>forceDeallocate</code>) returns 0. - <code>balanceOf</code> stays at 1, <code>setMorphoVault</code> reverts. The issue persists indefinitely unless external conditions change (e.g., the price recovers). The issue stems from using a share-based check (<code>balanceOf > 0</code>) instead of a value-based check. A nonzero share balance does not guarantee that the position has any realizable value. However, the system treats it as an active position regardless, leading to incorrect blocking behavior.
Recommendations	Consider replacing the share-based condition with a check based on redeemable assets. However, there is a trade-off: the vault may leave behind a small number of shares that could be redeemed once the price recovers. That said, implementing this recommendation would provide the option to either keep those shares or migrate to another vault.
Comments / Resolution	Resolved.

Issue_28	New Morpho vault setting is prevented via additional edge-cases
Severity	Low
Description	The issue occurs when the external Morpho position decreases in value while the vault still records a larger <code>adapterAllocated</code> amount. If the decrease exceeds <code>DEALLOCATE_BUFFER</code>, normal <code>deallocatable[vault]</code> returns zero. Since the position is underwater, <code>skimmable[vault]</code> also returns zero because <code>getAssets[vault] <= adapterAllocated</code>. As a result, neither normal <code>deallocation</code> nor <code>skim</code> can clear the remaining Morpho vault shares from the per-vault account. At the same time, <code>setMorphoVault()</code> checks only whether the account still holds shares of the current Morpho vault, so it reverts as long as any shares remain. The root cause is the interaction between the underwater guard in <code>deallocatable()</code> and the strict active-position check in <code>setMorphoVault()</code>. Impact 1: the curator can be unable to switch the vault to a new Morpho vault through normal flows while the current Morpho account still has shares Impact 2: the vault becomes dependent on curator <code>forceDeallocate()</code> or external <code>recover()</code> to unblock the configuration; Impact 3: if external Morpho liquidity is unavailable or external view calls revert, even <code>forceDeallocate()</code> may fail or return zero, extending the blocked state Impact 4: if <code>forceDeallocate()</code> clears the remaining shares but leaves stale <code>adapterAllocated</code>, the vault can switch to a new Morpho vault while old loss residue remains in adapter accounting.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_29	Loss of funds when allocating just before <code>MorphoVaultV2::liquidityAdapter</code> removal
Severity	Low
Description	If a user deposits into SymbioticVaultV2 and the first adapter in the array is the Morpho adapter, the funds will be automatically allocated to MorphoVaultV2. However, if a transaction to remove the liquidity adapter from Morpho Vault V2's whitelisted adapters is pending, those funds could still be allocated to that adapter and then become stuck once the adapter is removed. Morpho's documentation warns that caps should be set to zero before removing an adapter, but that does not eliminate the possibility of this scenario. See the following for the full impact of this issue: https://docs.morpho.org/curate/concepts/security-considerations/#adapter-removal-frontrunning <i>Note: the issue described in the documentation concerns an allocator front-running adapter removals, whereas here the concern is that users may unknowingly execute a transaction just before the liquidity adapter is removed.</i> Furthermore, even if the cap is set to zero, users may still interact with the liquidity adapter. In that case, allocation would revert and, as explained in other notes in this report, that could be the source of multiple issues due to failed allocation.
Recommendations	Consider keeping this in mind.
Comments / Resolution	Acknowledged. Lost funds can be covered by the <code>recover()</code> function.

Issue_30	<code>deallocatable</code> function does not represent available liquidity
Severity	Low
Description	The <code>deallocatable</code> function's purpose is to determine how much can be indeed deallocated from the third party protocol. The usage of <code>liquidityAdapter.realAssets</code> can be inaccurate because it does not reflect the available liquidity, as majority of the liquidity can be lent out which would result in a revert of the withdrawal.
Recommendations	Consider keeping this in mind.
Comments / Resolution	Acknowledged.

Issue_31	Liquidity within Morpho can be technically sufficient
Severity	Low
Description	Morpho vaults only deallocate from the <code>liquidityAdapter</code> but funds can also be relayed to other adapters. A <code>forceDeallocate</code> function exists to retrieve funds in such a situation but a fee will be applied for this execution. This is an inherent limitation for the <code>MorphoVaultV2Adapter</code> as it fully relies on funds within the <code>liquidityAdapter</code>.
Recommendations	Consider keeping this in mind. We do not recommend implementing a <code>forceDeallocate</code> call as the fee will disrupt the accounting.
Comments / Resolution	Acknowledged

Issue_32	MorphoVaultV2::liquidityAdapter misconfiguration concerns.
Severity	Low
Description	MorphoVaultV2::liquidityAdapter can be configured without being whitelisted as an adapter itself. In that case, SymbioticVaultV2 allocations and deallocations may be DoSed, because the system will attempt to allocate to and deallocate from MorphoVaultV2::liquidityAdapter. The following line will succeed: https://github.com/morpho-org/vault-v2/blob/b6c91d97d1129f5a87089c9c244fbcbbd74393bd/src/VaultV2.sol#L785 But inside allocateInternal will revert here: https://github.com/morpho-org/vault-v2/blob/b6c91d97d1129f5a87089c9c244fbcbbd74393bd/src/VaultV2.sol#L581 The same issue applies to deallocation. This misconfiguration also affects MorphoVaultV2Adapter::deallocatable(), because it only checks whether liquidityAdapter != address[0]. As a result, it may return an inflated deallocatable amount and later fail when attempting to deallocate more than is actually available.
Recommendations	Consider acknowledging this issue, while keeping in mind that this state may be possible
Comments / Resolution	Acknowledged

Issue_33	Slashing prevented due to negative yield event and shortfall larger than buffer
Severity	Informational
Description	<code>VaultV2.onSlash(amount, withAdapters)</code> reduces vault accounting first and optionally tries to recover collateral from adapters. The amount the vault wants back from adapters is driven by: <pre><code>_adaptersOwe[] = adaptersAllocated.saturatingSub[_maxAllocatable[]];</code></pre> where <code>_maxAllocatable[]</code> decreases when <code>activeStake</code> or <code>activeWithdrawals</code> are slashed. The Morpho adapter's normal deallocation refuses to deallocate when: <pre><code>allocated.saturatingSub(assets) > DEALLOCATE_BUFFER</code></pre> unless <code>_isForceDeallocate</code> is set. The slash path does not set <code>_isForceDeallocate</code>; only curator <code>forceDeallocate[]</code> does. When the external Morpho position suffers negative yield, fee drag, or share-price decrease, <code>getAssets(vault)</code> can fall below <code>adapterAllocated</code>. If the difference exceeds <code>DEALLOCATE_BUFFER</code>, normal <code>deallocatable(vault)</code> returns zero. During a slash, <code>VaultV2</code> reduces <code>activeStake</code> or withdrawals and increases <code>_adaptersOwe[]</code>, then calls normal <code>deallocateAdapters[]</code>. Since the Morpho adapter is underwater beyond the buffer, it returns zero and no collateral is recalled. The root cause is that slashing relies on normal adapter deallocation, while the Morpho adapter explicitly blocks normal deallocation in sufficiently underwater states. The emergency bypass exists only through curator-controlled <code>forceDeallocate[]</code>, which is not used by the slashing control-flow. This state can also be triggered artificially or organically.

Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_34	MorphoVaultV2::maxRate could deny full economic yield
Severity	Informational
Description	maxRate in Morphe can be used to cap the yield realized by the underlying vaults/markets. This helps keep the rate smooth and prevents sudden spikes in reported yield. In other words, maxRate may prevent the full yield earned by shareholders from being reflected immediately. As a result, if a Symbiotic::VaultV2 fully exits its position from MorphoVaultV2 while maxRate is still capping part of the yield, that unrealized portion will not be received by the exiting vault. When that yield is later realized, it will be accrued to the remaining MorphoVaultV2 shareholders.
Recommendations	This may be acceptable as a design choice, since that portion of the yield is technically not realized at the time of exit.
Comments / Resolution	Acknowledged

Issue_35	Risks inherited from underlying vaults/markets
Severity	Informational
Description	When funds are allocated to Morpho Vault V2, the resulting yield and losses ultimately depend on the underlying vaults and markets to which the Morpho adapter is exposed. Some Vault V1 and Vault V1.1 deployments may present risks that could be exploited. See the following for the relevant security considerations: https://docs.morpho.org/curate/concepts/security-considerations/
Recommendations	Consider keeping these risks and considerations in mind.
Comments / Resolution	Acknowledged

Issue_36	Adapter::forceDeallocate concerns
Severity	Informational
Description	<code>MorphoVaultV2Adapter</code> enforces an underwater guard inside <code>_deallocate</code>: When <code>adapterAllocated - getAssets > DEALLOCATE_BUFFER</code>, normal deallocations short-circuit to 0, so any role with <code>DEALLOCATE_ADAPTER_ROLE</code> cannot accidentally realize a loss that exceeds the configured buffer. The explicit channel for realizing such a loss is <code>forceDeallocate</code>, restricted to the curator; the actual extraction still goes through <code>morphoVault.withdraw(amount)</code>, which depends on Morpho's idle balance plus whatever its liquidity adapter cascade can resolve at that moment. Two independent mechanisms can degrade the share price of a Morpho VaultV2: - bad debt in an underlying market (drops <code>_totalAssets</code>) - management/performance fee accrual (inflates <code>totalSupply</code>). Either one alone, and especially both combined, can trigger a withdrawal stampede on the Morpho vault, depleting idle and forcing other depositors to invoke Morpho's <code>forceDeallocate</code> to extract from the cascaded adapters. By the time the Symbiotic curator wants to <code>forceDeallocate</code> to stop bleeding, the Morpho standard withdraw path may only be able to serve a small fraction of the requested amount. There is no alternative code path to extract the remainder. Morpho VaultV2 exposes its own <code>forceDeallocate()</code> primitive, which forces a specific adapter to push liquidity back to the vault's idle, paid for by burning shares from <code>onBehalf</code> according to a per-adapter penalty. The Symbiotic adapter cannot reach this primitive: <code>MorphoVaultV2Account</code> only exposes <code>withdraw(morphoVault, assets)</code> to the adapter, so neither the proxy nor the adapter can route a call to Morpho's force-deallocate. Even if the integration existed, the penalty is charged by burning the proxy's shares, which

	directly reduces the proxy's <code>getAssets</code> and so realizes additional loss on the Symbiotic side on top of the ongoing degradation – that trade-off (penalty cost vs. liquidity recovery) is the curator's call, but today they don't have the option.
Recommendations	This issue can be acknowledged. Alternatively, the adapter could integrate with <code>Morpho::forceDeallocate</code>, depending on its intended behavior. This would allow the curator (assumed to be trusted) to choose between penalty costs and liquidity recovery.
Comments / Resolution	Acknowledged.

AAVEV3Adapter

The **AAVEV3Adapter** contract is inheriting the **Adapter** contract and serves as a direct adapter for the **VaultV2** contract. Whenever a deposit or a manual allocation is happening, this contract will receive the corresponding amount of tokens and allocates these to the AAVEV3 pool (**Pool** contract). While abstracting away the specific control-flows, in a nutshell, is simply a deposit into a third party protocol where funds can be withdrawn again (under regular operations) and yield is accrued.

For the core mechanism, we refer to the same Appendix as within the **MorphoVaultV2Adapter** section.

Privileged Functions

- none

Core Invariants:

INV 1: Only the Vault can call allocate and deallocate

INV 2: Each Vault must be tied to a MorphoVault

INV 3: `_getVaultAssets` must return the token amount which is entitled to the VaultV2 based on its shares in context to the overall shares

INV 4: `_getAdapterAssets` must return all underlying assets for this adapter

Issue_37	Relying on <code>collateral.balanceOf[aToken]</code> could cause deallocations to fail
Severity	Low
Description	Inside <code>deallocatable()</code>, the function takes the minimum of the following two values: - <code>collateral.balanceOf[adapter] + min[_getVaultAssets[vault], collateral.balanceOf[aToken]]</code> - <code>adapterAllocated[adapter]</code> The issue is the reliance on <code>collateral.balanceOf[aToken]</code>, because in some cases it does not reflect the actual amount that can be withdrawn and may be slightly inflated. Aave V3-origin uses a virtual underlying balance rather than the raw balance, and some <code>aToken</code> contracts already show a discrepancy between the raw balance and the virtual balance. For example, in the <code>aUSDC</code> contract, the discrepancy is approximately <code>6,401 USDC</code>. Aave pool (to check the virtual balance): - https://etherscan.io/address/0x87870Bca3F3fD6335C3F4ce8392D69350B4fA4E2#readProxyContract <code>aUSDC</code> contract: - https://etherscan.io/address/0x98C23E9d8f34FEFb1B7BD6a91B7FF122F4e16F5c#readProxyContract <code>USDC</code> contract (to check the raw balance) - https://etherscan.io/address/0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48#readProxyContract In <code>aUSDT</code> contract the discrepancy is around <code>~1455</code> tokens. This means that when markets for <code>aTokens</code> with such discrepancies become heavily utilized, close to 100% utilization, as has already happened with the <code>USDT</code> market, an adapter attempting to deallocate funds may try to withdraw an inflated amount, causing <code>Aave.withdraw()</code> to revert. Although those reverts are caught through a fallback catch, that

	does not necessarily mean the overall transaction will succeed. For example, if a user is trying to claim a withdrawal, <code>VaultV2::deallocateAdapters()</code> may not withdraw enough funds to fulfill the withdrawal due to this issue and the claim transaction could still revert due to insufficient funds. The same issue applies to <code>instantWithdrawal</code>. In that case, <code>instantWithdrawal</code> may revert even earlier because of the following check: <pre>if (_maxAllocatable() < adaptersAllocated) { revert InsufficientAmount(); }</pre>
Recommendations	Consider querying the virtual balance directly, either using: - <code>IPool(pool).getVirtualUnderlyingBalance(token);</code> - <code>IPool(pool).getReserveDataExtended(token).virtualUnderlyingBalance;</code>
Comments / Resolution	Resolved.